Corrigés commentés de la semaine Git

Tous les exercices de la semaine, du lundi au jeudi, plus les exercices de publication, l'enquête et le festival. Le TP du vendredi n'y est pas.

Comment lire ce corrigé. Essayez d'abord seuls, ou relisez votre propre travail. Ensuite seulement, comparez. Pour chaque exercice, vous trouverez les commandes, ce que Git répond, et surtout **pourquoi**. Les encadrés « Piège » signalent les erreurs qu'on voit le plus souvent.

Les terminaux sont réels. Toutes les sorties viennent d'une vraie exécution, avec Git 2.43 comme sur votre VM Ubuntu. L'utilisatrice des exemples s'appelle Alice Martin, son pseudo GitHub est `alice-martin`, et son dossier s'appelle `/home/robin/demo`. Chez vous, trois choses seront différentes :

- les **identifiants de commit** (`581d41b`…) : ils dépendent du nom de l'auteur, de la date et du contenu, donc ils sont différents chez chacun ;
- les **messages de GitHub** (lignes qui commencent par `remote:`) : ici, GitHub est simulé par un dossier, et certaines lignes que GitHub ajoute n'apparaissent pas ;
- l'**éditeur** : ici, les messages de fusion sont acceptés automatiquement. Chez vous, nano s'ouvre : gardez le message proposé, `Ctrl+O`, `Entrée`, `Ctrl+X`.

LUNDI · GIT SUR VOTRE MACHINE

L1. Configurer Git

Les cinq commandes de la section 1, une seule fois par machine :

```
git config --global user.name "Alice Martin"
git config --global user.email "alice@example.com"
git config --global init.defaultBranch main
git config --global core.editor nano
git config --global pull.rebase false
```

Terminal

```
$ git config --global --list
user.name=Alice Martin
user.email=alice@example.com
init.defaultbranch=main
core.editor=nano
pull.rebase=false
```

Chaque ligne a son utilité :

- `user.name` et `user.email` signent chacun de vos commits. Mettez la **même adresse que sur GitHub**, sinon GitHub ne saura pas que les commits sont à vous.
- `init.defaultbranch=main` : chaque nouveau dépôt commence sur une branche `main` (l'ancien nom était `master`).
- `core.editor=nano` : quand Git a besoin d'un message, il ouvre nano au lieu de vim.
- `pull.rebase=false` : `git pull` fusionne quand c'est nécessaire, au lieu de s'arrêter sur une erreur. On en a eu besoin mercredi.

Une faute de frappe ? Relancez la même commande avec la bonne valeur : elle remplace l'ancienne. Pour voir une seule valeur : `git config user.email`.

L2. Versionner vos scripts bash

Le déroulé complet

```
$ mkdir algo-bash && cd algo-bash
$ git init
Initialized empty Git repository in /home/robin/demo/algo-bash/.git/
$ ls
bonjour.sh
pair.sh
table.sh
$ git status --short
?? bonjour.sh
?? pair.sh
?? table.sh
$ git add .
$ git commit -m "Ajoute mes exercices de bash"
[main (root-commit) 581d41b] Ajoute mes exercices de bash
 3 files changed, 6 insertions(+)
 create mode 100644 bonjour.sh
 create mode 100644 pair.sh
 create mode 100644 table.sh
$ git log --oneline
581d41b (HEAD -> main) Ajoute mes exercices de bash
```

Lisons ce que Git répond :

- `Initialized empty Git repository in …/.git/` : Git a créé le dossier caché `.git`. C'est **lui** le dépôt : tout l'historique est dedans. Si vous le supprimez, l'historique disparaît, mais vos fichiers restent.
- `??` devant chaque script : fichier **non suivi**. Git le voit, mais ne s'en occupe pas encore.
- `(root-commit)` : c'est le premier commit du dépôt, la racine de l'historique.
- `3 files changed, 6 insertions(+)` : trois fichiers, six lignes ajoutées.

> **Piège.** Lancer `git init` dans votre home au lieu du dossier `algo-bash`. Symptôme : `git status` affiche des centaines de fichiers. Vérifiez où vous êtes avec `pwd` avant chaque `git init`.

L3. Un commit par changement

Deux modifications sans rapport : un commentaire dans `table.sh`, un nouveau message dans `bonjour.sh`.

Terminal

```
$ git status --short
 M bonjour.sh
 M table.sh
$ git add table.sh
$ git commit -m "Ajoute un commentaire à table.sh"
[main 52b3913] Ajoute un commentaire à table.sh
 1 file changed, 1 insertion(+)
$ git add bonjour.sh
$ git commit -m "Change le message de bonjour.sh"
[main 0c8be65] Change le message de bonjour.sh
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline --stat
0c8be65 (HEAD -> main) Change le message de bonjour.sh
 bonjour.sh | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
52b3913 Ajoute un commentaire à table.sh
 table.sh | 1 +
 1 file changed, 1 insertion(+)
581d41b Ajoute mes exercices de bash
 bonjour.sh | 2 ++
 pair.sh    | 2 ++
 table.sh   | 2 ++
 3 files changed, 6 insertions(+)
```

`M` veut dire : modifié, pas encore dans l'index. Le `git add` d'un seul fichier à la fois fait le tri : chaque commit ne contient que ce qui a été ajouté juste avant.

Pourquoi se donner ce mal ? Parce qu'un commit qui fait une seule chose est facile à comprendre, à relire, et à annuler sans abîmer le reste. L'enquête de lundi soir l'a montré : un commit qui mélangeait deux changements a rendu la réparation pénible.

> **Piège.** `git add .` ou `git commit -am` auraient mis les deux modifications dans le même commit.

L4. Le détective

Terminal

```
$ git log --oneline | wc -l
3
$ git show --stat HEAD~1
commit 52b3913197a3f3025ff8cd9be25648e7b303b76b
Author: Alice Martin <alice@example.com>
Date:   Mon Oct 26 10:00:00 2026 +0100

    Ajoute un commentaire à table.sh

 table.sh | 1 +
 1 file changed, 1 insertion(+)
$ git diff HEAD~2 HEAD
diff --git a/bonjour.sh b/bonjour.sh
index 445f9aa..e3a9c01 100644
--- a/bonjour.sh
+++ b/bonjour.sh
@@ -1,2 +1,2 @@
 #!/bin/bash
-echo bonjour
+echo bonjour depuis Git
diff --git a/table.sh b/table.sh
index 4161543..f055ae2 100644
--- a/table.sh
+++ b/table.sh
@@ -1,2 +1,3 @@
 #!/bin/bash
+# Affiche la table de multiplication du nombre donné
 for (( i = 1; i <= 10; i++ )); do echo "$1 x $i = $(( $1 * i ))"; done
```

1. **3 commits.** `wc -l` compte les lignes, et `git log --oneline` affiche une ligne par commit.
2. Le deuxième commit n'a modifié que `table.sh` (une ligne ajoutée).
3. Entre le premier et le dernier commit : le commentaire de `table.sh` et le message de `bonjour.sh`.

Au lieu de recopier des identifiants, on peut compter à partir du dernier commit : `HEAD` est le dernier, `HEAD~1` l'avant-dernier, `HEAD~2` celui d'avant. Avec des identifiants, `git diff 581d41b 0c8be65` donne exactement le même résultat.

Comment lire un `diff` : une ligne qui commence par `-` a été retirée, une ligne qui commence par `+` a été ajoutée, les autres sont là pour le contexte. Une ligne modifiée apparaît donc deux fois : l'ancienne version en `-`, la nouvelle en `+`.

L5. Le fichier .gitignore

Terminal

```
$ git status --short
?? brouillon.txt
?? test.log
?? tmp/
$ printf "brouillon.txt\n*.log\ntmp/\n" > .gitignore
$ cat .gitignore
brouillon.txt
*.log
tmp/
$ git status --short
?? .gitignore
$ git add .gitignore && git commit -m "Ignore les brouillons, les logs et tmp"
[main 3a6e0db] Ignore les brouillons, les logs et tmp
 1 file changed, 3 insertions(+)
 create mode 100644 .gitignore
```

Les trois règles :

- `brouillon.txt` : ce fichier précis ;
- `*.log` : tous les fichiers qui finissent par `.log`, où qu'ils soient. C'est mieux que `test.log`, qui n'ignorerait que celui-là ;
- `tmp/` : le dossier entier et tout ce qu'il contient.

Vous pouviez aussi écrire le `.gitignore` avec nano, une règle par ligne. Le `.gitignore` lui-même **doit** être commité : il fait partie du projet, et toute l'équipe en profite.

> **Piège.** Le `.gitignore` ne s'applique qu'aux fichiers **pas encore suivis**. Un fichier déjà commité reste suivi. Pour l'arrêter : `git rm --cached fichier`, puis commit. Le fichier reste sur votre disque, mais Git l'oublie.

L6. Réparer ses erreurs

1. Le fichier écrasé

```
$ echo "oups" > pair.sh
$ git restore pair.sh
$ cat pair.sh
#!/bin/bash
echo "$1 est pair"
```

`git restore` remet le fichier tel qu'il était au dernier commit. Attention, c'est sans retour : la modification annulée est perdue.

2. Le add de trop

```
$ echo "# TODO" >> table.sh
$ git add table.sh
$ git restore --staged table.sh
$ git status --short
 M table.sh
$ tail -n 1 table.sh
# TODO
```

`--staged` sort le fichier de l'index, sans toucher à son contenu : la ligne `# TODO` est toujours là, et `M` confirme que le fichier est modifié mais plus prêt à partir.

3. Le commit à annuler

```
$ echo 'echo "bug"' >> bonjour.sh
$ git commit -qam "Ajoute un message de debug"
$ git revert --no-edit HEAD
[main d7d897b] Revert "Ajoute un message de debug"
 Date: Mon Oct 26 10:00:00 2026 +0100
 1 file changed, 1 deletion(-)
$ git log --oneline
d7d897b (HEAD -> main) Revert "Ajoute un message de debug"
c3ed07d Ajoute un message de debug
3a6e0db Ignore les brouillons, les logs et tmp
0c8be65 Change le message de bonjour.sh
52b3913 Ajoute un commentaire à table.sh
581d41b Ajoute mes exercices de bash
```

Réponse : deux commits de plus. Le commit fautif, **et** le commit qui l'annule. `git revert` n'efface rien : il crée un nouveau commit qui fait exactement l'inverse. L'historique garde la trace de l'erreur et de sa correction. C'est la bonne méthode dès que le commit est partagé sur GitHub, puisque personne ne perd rien.

Ici, `--no-edit` garde le message proposé (`Revert "..."`). Sans lui, nano s'ouvre avec ce même message.

L7. Chercher dans l'historique

Terminal

```
$ git log -S 'echo "bug"' --oneline
d7d897b (HEAD -> main) Revert "Ajoute un message de debug"
c3ed07d Ajoute un message de debug
$ git blame bonjour.sh
^581d41b (Alice Martin 2026-10-26 10:00:00 +0100 1) #!/bin/bash
0c8be651 (Alice Martin 2026-10-26 10:00:00 +0100 2) echo bonjour depuis Git
```

`git log -S` trouve **les deux** commits : `c3ed07d` a ajouté le texte, `d7d897b` l'a retiré. Il cherche les commits qui changent le nombre d'apparitions du texte, dans un sens ou dans l'autre.

`git blame` affiche, pour chaque ligne : le commit, l'auteur, la date, puis la ligne. Le `^` devant `581d41b` signale le tout premier commit. Les apostrophes autour de `'echo "bug"'` évitent que bash ne mange les guillemets doubles.

MARDI · BRANCHES, CONFLITS ET GITHUB

M1. Une branche pour une nouvelle fonctionnalité

Terminal

```
$ git switch -c ajout-rebours
Switched to a new branch 'ajout-rebours'
$ git add rebours.sh && git commit -m "Ajoute le compte à rebours"
[ajout-rebours b719587] Ajoute le compte à rebours
 1 file changed, 3 insertions(+)
 create mode 100644 rebours.sh
$ git switch main
Switched to branch 'main'
$ ls
bonjour.sh
brouillon.txt
pair.sh
table.sh
test.log
tmp
$ git merge ajout-rebours
Updating d7d897b..b719587
Fast-forward
 rebours.sh | 3 +++
 1 file changed, 3 insertions(+)
 create mode 100644 rebours.sh
$ ls
bonjour.sh
brouillon.txt
pair.sh
rebours.sh
table.sh
test.log
tmp
$ git branch -d ajout-rebours
Deleted branch ajout-rebours (was b719587).
```

Sur `main`, `rebours.sh` a disparu : le fichier n'existe que dans le commit de la branche. Changer de branche remet le dossier dans l'état de cette branche. Rien n'est perdu, il attend sur sa branche.

`Fast-forward` : `main` n'avait pas bougé depuis la création de la branche. Git a juste avancé `main` jusqu'au commit de la branche, sans créer de commit de fusion. Le `-d` minuscule refuse de supprimer une branche qui n'est pas fusionnée : c'est une sécurité.

M2. Deux branches qui divergent

Terminal

```
$ git switch -c commentaires
Switched to a new branch 'commentaires'
$ sed -i '1a # Dit si un nombre est pair' pair.sh
$ git commit -qam "Commente pair.sh"
$ git switch main
Switched to branch 'main'
$ echo 'echo "fin"' >> rebours.sh
$ git commit -qam "Ajoute un message de fin au compte à rebours"
$ git merge commentaires
Merge made by the 'ort' strategy.
 pair.sh | 1 +
 1 file changed, 1 insertion(+)
$ git log --oneline --graph
*   73d286c (HEAD -> main) Merge branch 'commentaires'
|\
| * 58e97c2 (commentaires) Commente pair.sh
* | 5e5d77c Ajoute un message de fin au compte à rebours
|/
* b719587 Ajoute le compte à rebours
* d7d897b Revert "Ajoute un message de debug"
* c3ed07d Ajoute un message de debug
* 3a6e0db Ignore les brouillons, les logs et tmp
* 0c8be65 Change le message de bonjour.sh
* 52b3913 Ajoute un commentaire à table.sh
* 581d41b Ajoute mes exercices de bash
```

Pourquoi un message cette fois ? En M1, `main` n'avait pas avancé : un simple fast-forward suffisait. Ici, `main` et `commentaires` ont chacune un commit que l'autre n'a pas. Git doit créer un **commit de fusion** qui réunit les deux, et tout commit a besoin d'un message. Chez vous, nano s'est ouvert avec `Merge branch 'commentaires'` : on garde, on enregistre, on ferme.

Dans le graphe, la bifurcation se voit : les deux branches partent de `b719587` et se rejoignent dans `73d286c`. Pas de conflit, puisque les deux branches ne touchaient pas le même fichier.

M3. Le conflit volontaire

Terminal

```
$ echo "Auteur : à compléter" > auteur.txt && git add auteur.txt && git commit -qm "Ajoute
auteur.txt"
$ git switch -c prenom
Switched to a new branch 'prenom'
$ echo "Auteur : Alice" > auteur.txt && git commit -qam "Ajoute le prénom"
$ git switch main
Switched to branch 'main'
$ echo "Auteur : Martin" > auteur.txt && git commit -qam "Ajoute le nom"
$ git merge prenom
Auto-merging auteur.txt
CONFLICT (content): Merge conflict in auteur.txt
Automatic merge failed; fix conflicts and then commit the result.
$ cat auteur.txt
<<<<<<< HEAD
Auteur : Martin
=======
Auteur : Alice
>>>>>>> prenom
```

Les deux branches ont changé **la même ligne**, de deux façons différentes. Git ne peut pas choisir à votre place : il s'arrête et écrit les deux versions dans le fichier.

- entre `<<<<<<< HEAD` et `=======` : la version de la branche où vous êtes (`main`) ;
- entre `=======` et `>>>>>>> prenom` : la version de la branche que vous fusionnez.

On ouvre le fichier avec nano, on écrit la version finale, on supprime les trois lignes de marqueurs, puis on termine la fusion :

Terminal

```
$ echo "Auteur : Alice Martin" > auteur.txt
$ cat auteur.txt
Auteur : Alice Martin
$ git add auteur.txt && git commit --no-edit
[main 20880a2] Merge branch 'prenom'
$ git log --oneline --graph -4
*   20880a2 (HEAD -> main) Merge branch 'prenom'
|\
| * e63e252 (prenom) Ajoute le prénom
* | 0b3bcfa Ajoute le nom
|/
* d8aa030 Ajoute auteur.txt
```

Le `git add` dit à Git « ce conflit est réglé ». Le `git commit` termine la fusion. Si vous êtes perdu au milieu, `git merge --abort` remet tout comme avant la fusion.

> **Piège.** Oublier un marqueur dans le fichier. Git ne vérifie pas : il commitera `=======` avec le reste. Avant le `git add`, un `cat` du fichier, ou `grep -n "=======" auteur.txt`.

M4. Compte GitHub et clé SSH

Pas de terminal à comparer ici : c'est réussi quand cette commande vous salue par votre pseudo.

```
ssh-keygen -t ed25519 -C "alice@example.com"     # Entrée à chaque question
cat ~/.ssh/id_ed25519.pub                         # la clé PUBLIQUE, à copier
ssh -T git@github.com                             # yes la première fois
Hi alice-martin! You've successfully authenticated, but GitHub does not provide shell access.
```

Entre la deuxième et la troisième commande, sur GitHub : photo en haut à droite, **Settings**, **SSH and GPG keys**, **New SSH key**, collez la ligne qui commence par `ssh-ed25519`, **Add SSH key**.

Ce que vous voyez	Ce que ça veut dire
`Hi pseudo! You've successfully authenticated…`	c'est bon, le « does not provide shell access » est normal
`Permission denied (publickey)`	la clé n'est pas sur GitHub, ou vous avez collé la clé privée (le fichier sans `.pub`)
`Are you sure you want to continue connecting`	première connexion : tapez `yes` en entier

M5. Publier `algo-bash`

Sur GitHub : **New**, nom `algo-bash`, **sans rien cocher**, **Create repository**, onglet **SSH**. Puis dans votre dépôt local :

Terminal

```
$ git remote add origin git@github.com:alice-martin/algo-bash.git
$ git remote -v
origin  git@github.com:alice-martin/algo-bash.git (fetch)
origin  git@github.com:alice-martin/algo-bash.git (push)
$ git push -u origin main
To git@github.com:alice-martin/algo-bash.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
$ git log --oneline | wc -l
14
```

`git remote -v` vérifie l'adresse avant d'envoyer. `[new branch] main -> main` confirme que la branche est arrivée sur GitHub. Sur la page du dépôt, l'horloge affiche **14 Commits** : le même nombre que `git log --oneline | wc -l`. Votre historique complet est parti, pas seulement les fichiers.

M6. Un vrai README

Un exemple qui répond aux quatre consignes, écrit avec `nano README.md` :

Terminal

```
$ cat README.md
# Algo bash

Mes exercices d'algorithmique en bash, écrits pendant la formation Prépavenir (octobre 2026).

## Lancer un script

Chaque script se lance avec bash, en lui donnant ses arguments :

```bash
bash table.sh 7
```

Les scripts

- `bonjour.sh` : affiche un message de bienvenue
- `pair.sh` : dit si un nombre est pair
- `table.sh` : affiche la table de multiplication du nombre donné
- `rebours.sh` : compte à rebours à partir du nombre donné

Autrice

Alice Martin
$ git add README.md
$ git commit -m "Ajoute le README"
[main 4087766] Ajoute le README
 1 file changed, 22 insertions(+)
 create mode 100644 README.md
$ git push
To git@github.com:alice-martin/algo-bash.git
   20880a2..4087766 main -> main
```

Sur GitHub, le README s'affiche mis en forme sous la liste des fichiers. Les titres `#` deviennent des titres, la liste devient une liste à puces, le bloc de code est sur fond gris.

> **Piège.** Modifier le README sur GitHub (le crayon), puis commiter sur votre VM sans `git pull` : votre prochain `git push` est refusé avec `(fetch first)`. GitHub a un commit que vous n'avez pas. Faites `git pull`, puis `git push`.

M7. Chez le voisin

```
git clone git@github.com:bob-dupont/algo-bash.git algo-bash-bob
cd algo-bash-bob
git log --oneline | wc -l          # combien de commits
git shortlog -sn HEAD              # qui en est l'auteur
```

Le dernier mot du `git clone` donne le nom du dossier : sans lui, Git l'appellerait `algo-bash`, comme votre propre dépôt. Clonez-le **à côté** de votre dépôt, jamais dedans : un dépôt dans un dépôt, c'est la confusion assurée.

Le clone et le commit marchent. Le `git push`, lui, est refusé :

```
ERROR: Permission to bob-dupont/algo-bash.git denied to alice-martin.
fatal: Could not read from remote repository.
```

Pourquoi ? Tout le monde peut **lire** un dépôt public, mais seuls son propriétaire et les personnes qu'il a invitées peuvent y **écrire**. Votre commit reste sur votre machine, GitHub refuse de le recevoir. Les deux solutions, vues mercredi : être invité comme collaborateur, ou proposer ses changements par une pull request.

EXERCICES DE PUBLICATION · CRÉER, ORGANISER, PUBLIER

G1. Bonjour GitHub

Terminal

```
$ mkdir bonjour-github && cd bonjour-github
$ git init
Initialized empty Git repository in /home/robin/demo/bonjour-github/.git/
$ cat README.md
# Bonjour GitHub

Je suis Alice, en formation de développeuse chez Prépavenir.
$ bash bonjour.sh
Bonjour depuis GitHub
$ git add .
$ git commit -m "Ajoute le README et le script bonjour"
[main (root-commit) d47bf50] Ajoute le README et le script bonjour
 2 files changed, 5 insertions(+)
 create mode 100644 README.md
 create mode 100644 bonjour.sh
$ git remote add origin git@github.com:alice-martin/bonjour-github.git
$ git push -u origin main
To git@github.com:alice-martin/bonjour-github.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Sur GitHub : deux fichiers, le README affiché en dessous, et **1 Commit**. Puis la deuxième modification :

Terminal

```
$ echo "J'apprends Git et GitHub cette semaine." >> README.md
$ git commit -am "Complète la présentation"
[main 2edef7f] Complète la présentation
 1 file changed, 1 insertion(+)
$ git push
To git@github.com:alice-martin/bonjour-github.git
   d47bf50..2edef7f  main -> main
```

Cette fois, pas de `-u origin main` : depuis le premier push, la branche `main` locale sait qu'elle est reliée à `origin/main`. La ligne `ancien..nouveau main -> main` montre l'avancée sur GitHub : du dernier commit qu'il connaissait au nouveau. Sur la page, **2 Commits**.

G2. Le carnet de bord et le mystère des dossiers vides

Ce qu'on remarque à l'étape 2 : `git status` ne montre que `README.md`. Les cinq dossiers existent sur le disque, mais ils sont vides, et Git ne suit que des **fichiers**. Pour lui, un dossier n'est qu'une partie du chemin d'un fichier : sans fichier dedans, il n'existe pas.

Le carnet complet, publié

```
$ mkdir carnet-de-bord && cd carnet-de-bord && git init -q
$ mkdir lundi mardi mercredi jeudi vendredi
$ echo "# Carnet de bord" > README.md
$ git status --short
?? README.md
$ touch mercredi/.gitkeep jeudi/.gitkeep vendredi/.gitkeep
$ git status --short
?? README.md
?? jeudi/
?? lundi/
?? mardi/
?? mercredi/
?? vendredi/
$ git add .
$ git commit -m "Crée le carnet de bord, un dossier par jour"
[main (root-commit) 3e4571a] Crée le carnet de bord, un dossier par jour
 6 files changed, 9 insertions(+)
 create mode 100644 README.md
 create mode 100644 jeudi/.gitkeep
 create mode 100644 lundi/notes.md
 create mode 100644 mardi/notes.md
 create mode 100644 mercredi/.gitkeep
 create mode 100644 vendredi/.gitkeep
$ git ls-files
README.md
jeudi/.gitkeep
lundi/notes.md
mardi/notes.md
mercredi/.gitkeep
vendredi/.gitkeep
$ git remote add origin git@github.com:alice-martin/carnet-de-bord.git
$ git push -u origin main
To git@github.com:alice-martin/carnet-de-bord.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Une fois remplis, les dossiers apparaissent : `git status --short` les affiche en entier (`?? lundi/`) parce que tout leur contenu est nouveau. `git ls-files` liste les six fichiers suivis : c'est exactement ce qui sera sur GitHub. Les `.gitkeep` commencent par un point, ils sont donc cachés par un simple `ls` (`ls -a` les montre), mais GitHub les affiche.

G3. Réorganiser sans perdre l'historique

La réponse à la question 4, sur un dépôt où `pair.sh` a été modifié avant d'être rangé :

Terminal

```
$ mkdir lundi mardi tp-pendu
$ git mv pair.sh table.sh lundi/
$ git mv moyenne.sh mardi/
$ git mv pendu.sh tp-pendu/
$ git commit -qm "Range les scripts par jour"
$ git log --oneline -- lundi/pair.sh
a1b8c24 (HEAD -> main) Range les scripts par jour
$ git log --oneline --follow -- lundi/pair.sh
a1b8c24 (HEAD -> main) Range les scripts par jour
7dd2a55 Commente pair.sh
0d93f81 Ajoute mes exercices de bash
```

Réponse : sans `--follow`, Git ne cherche que le chemin `lundi/pair.sh`, qui n'existe que depuis le rangement : un seul commit. Avec `--follow`, il comprend que `lundi/pair.sh` s'appelait `pair.sh` avant, et remonte toute l'histoire du fichier : trois commits. Rien n'a été perdu en déplaçant.

Avec un simple `mv` suivi de `git add -A`, Git aurait fini par reconnaître le déplacement, parce que le contenu est identique. `git mv` le fait en une commande, et `git status` affiche directement `renamed:`.

Le README à jour, avec son tableau, puis l'envoi :

```
## Organisation

| Dossier | Contenu |
|---|---|
| lundi/ | variables, conditions, boucles |
| mardi/ | fonctions, tableaux, chaînes |
| tp-pendu/ | le jeu du pendu |
```

```
git add README.md
git commit -m "Décrit la nouvelle organisation dans le README"
git push
```

Le `git push` envoie d'un coup les deux commits : le rangement et le README.

G4. L'aller-retour avec GitHub

Étapes 1 à 3. La modification faite sur GitHub (le crayon, puis **Commit changes**) crée un commit sur GitHub, avec le message proposé par défaut, `Update README.md`. On la récupère :

Terminal

```
$ git pull
From github.com:alice-martin/bonjour-github
   2edef7f..62f3eea  main       -> origin/main
Updating 2edef7f..62f3eea
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
$ tail -n 2 README.md
J'apprends Git et GitHub cette semaine.
Ligne ajoutée depuis GitHub.
$ echo 'echo "À bientôt"' >> bonjour.sh
$ git commit -am "Ajoute un au revoir à bonjour.sh"
[main 90b843e] Ajoute un au revoir à bonjour.sh
 1 file changed, 1 insertion(+)
$ git push
To git@github.com:alice-martin/bonjour-github.git
   62f3eea..90b843e  main -> main
```

`Fast-forward` : vous n'aviez rien fait de votre côté, Git a juste avancé votre `main`.

Étapes 4 à 6, le conflit. Sur GitHub, la première ligne devient `# Bonjour GitHub, version web`. Sur la VM, sans `git pull`, elle devient `# Bonjour GitHub, version VM` :

Terminal

```
$ sed -i '1s/.*/# Bonjour GitHub, version VM/' README.md
$ git commit -qam "Change le titre"
$ git push
To git@github.com:alice-martin/bonjour-github.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'git@github.com:alice-martin/bonjour-github.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
$ git pull
From github.com:alice-martin/bonjour-github
   90b843e..fa99275  main       -> origin/main
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
$ head -n 5 README.md
<<<<<<< HEAD
# Bonjour GitHub, version VM
=======
# Bonjour GitHub, version web
>>>>>>> fa9927540368e9002e4cb69695c163b3681d4510
```

Le push est refusé avec `(fetch first)` : GitHub a un commit que vous n'avez pas. Le `git pull` récupère ce commit et tente la fusion, mais les deux commits touchent la même ligne : conflit, exactement comme mardi matin. On écrit le titre définitif avec nano (ici, on revient à `# Bonjour GitHub`), on supprime les trois marqueurs, et on termine :

Terminal

```
$ head -n 1 README.md
# Bonjour GitHub
$ git add README.md
$ git commit --no-edit
[main de482fa] Merge branch 'main' of github.com:alice-martin/bonjour-github
$ git push
To git@github.com:alice-martin/bonjour-github.git
   fa99275..de482fa  main -> main
```

G5. Le clone de contrôle

Terminal

```
$ git clone git@github.com:alice-martin/carnet-de-bord.git verif-carnet
Cloning into 'verif-carnet'...
done.
$ cd verif-carnet
$ ls
README.md
jeudi
lundi
mardi
mercredi
vendredi
$ git log --oneline
3e4571a (HEAD -> main, origin/main, origin/HEAD) Crée le carnet de bord, un dossier par jour
$ echo "- git push -u, git pull" >> mardi/notes.md
$ git commit -am "Complète les notes de mardi"
[main 2fc2008] Complète les notes de mardi
 1 file changed, 1 insertion(+)
$ git push
To git@github.com:alice-martin/carnet-de-bord.git
   3e4571a..2fc2008  main -> main
$ cd ../carnet-de-bord
$ git pull
From github.com:alice-martin/carnet-de-bord
   3e4571a..2fc2008  main       -> origin/main
Updating 3e4571a..2fc2008
Fast-forward
 mardi/notes.md | 1 +
 1 file changed, 1 insertion(+)
$ tail -n 2 mardi/notes.md
- GitHub et la clé SSH
- git push -u, git pull
```

Le clone contient tout : les fichiers **et** l'historique. Si un fichier manque dans `verif-carnet`, c'est qu'il n'a jamais été commité, ou jamais poussé. C'est le meilleur test avant de rendre un travail.

Deux dossiers sur la même machine, synchronisés par GitHub : c'est exactement la situation de deux personnes d'une équipe. Chacun pousse ses commits, et récupère ceux de l'autre avec `git pull`.

G6. Votre profil GitHub

Un exemple de README de profil, dans le dépôt `alice-martin/alice-martin` :

```
# Bonjour, je suis Alice 👋

Après dix ans dans la logistique, je me reconvertis dans le développement.
Je suis la formation Prépavenir depuis septembre 2026.

## Ce que je sais faire

- Linux : terminal, droits, SSH
- Bash : scripts, boucles, fonctions, tableaux
- Git et GitHub : branches, conflits, pull requests

## Mes projets

- [algo-bash](https://github.com/alice-martin/algo-bash) : mes exercices d'algorithmique en bash
- [carnet-de-bord](https://github.com/alice-martin/carnet-de-bord) : mon carnet de formation
```

Les points à vérifier : le dépôt porte **exactement** votre pseudo (majuscules comprises), il est **public**, et le fichier s'appelle `README.md` à la racine. Un recruteur lira ce texte en 30 secondes : restez court, concret, et mettez des liens vers vos meilleurs dépôts.

G7. Le piège du README

Terminal

```
$ mkdir piege && cd piege && git init -q
$ git add . && git commit -qm "Premier commit"
$ git remote add origin git@github.com:alice-martin/piege.git
$ git push -u origin main
To git@github.com:alice-martin/piege.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'git@github.com:alice-martin/piege.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
$ git pull origin main
From github.com:alice-martin/piege
 * branch            main       -> FETCH_HEAD
 * [new branch]      main       -> origin/main
fatal: refusing to merge unrelated histories
$ git pull origin main --allow-unrelated-histories
From github.com:alice-martin/piege
 * branch            main       -> FETCH_HEAD
Auto-merging README.md
CONFLICT (add/add): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
$ cat README.md
<<<<<<< HEAD
# Mon projet
=======
# piege
>>>>>>> 36bc4e8fdd078ca0b10574ba77098147b77563b7
```

Les trois refus, dans l'ordre :

1. `(fetch first)` : GitHub a un commit (`Initial commit`, son README) que vous n'avez pas.
2. `refusing to merge unrelated histories` : vos deux historiques n'ont **aucun commit en commun**. Ce ne sont pas deux versions d'un même projet, mais deux projets qui portent le même nom. Git refuse de les marier sans votre accord.
3. `CONFLICT (add/add)` : avec l'accord, la fusion se fait, mais les deux côtés ont **créé** un `README.md` différent.

On garde notre README, on termine la fusion, on pousse :

Terminal

```
$ git add README.md
$ git commit --no-edit
[main 92a6403] Merge branch 'main' of github.com:alice-martin/piege
$ git push -u origin main
To git@github.com:alice-martin/piege.git
   36bc4e8..92a6403  main -> main
branch 'main' set up to track 'origin/main'.
$ git log --oneline --graph
*   92a6403 (HEAD -> main, origin/main) Merge branch 'main' of github.com:alice-martin/piege
|\
| * 36bc4e8 Initial commit
* 3bf7567 Premier commit
```

Le graphe montre deux racines qui se rejoignent : la marque d'une fusion d'historiques sans rapport.

La phrase demandée : on crée le dépôt GitHub vide parce que, s'il contient déjà un commit, il a son propre historique, sans lien avec le nôtre, et il faut alors forcer une fusion qui finit en conflit. Vide, il accepte notre historique tel quel.

G8. Un site en ligne avec GitHub Pages

```
mkdir mon-site && cd mon-site
git init
nano index.html                    # le code de l'énoncé, avec votre prénom
git add index.html
git commit -m "Ajoute la page d'accueil"
git remote add origin git@github.com:alice-martin/mon-site.git
git push -u origin main
```

Puis **Settings**, **Pages**, *Deploy from a branch*, `main`, `/ (root)`, **Save**. Une à deux minutes plus tard, l'adresse `https://alice-martin.github.io/mon-site/` s'affiche en haut de la page des réglages. Pendant la mise en ligne, l'onglet **Actions** montre une tâche *pages build and deployment* : quand elle est verte, le site est en ligne.

Si ça ne marche pas, vérifiez trois choses :

- le fichier s'appelle exactement `index.html`, en minuscules, à la racine du dépôt (pas dans un sous-dossier) ;
- le dépôt est **public** : avec un compte gratuit, GitHub Pages ne fonctionne pas sur un dépôt privé ;
- une page 404 juste après la mise en route : attendez deux minutes et rechargez.

Chaque `git push` sur `main` republie le site automatiquement. C'est déjà du **déploiement continu** : le code poussé arrive en ligne sans manipulation.

MERCREDI · TRAVAILLER À PLUSIEURS

Pour ces corrigés, l'équipe s'appelle Alice, Bob et Chloé. Alice a créé le dépôt `equipe-scripteurs` et invité les deux autres. Sauf mention contraire, le terminal est celui de Bob.

Me1. Le dépôt d'équipe

Côté Alice, sur GitHub :

1. **New**, nom `equipe-scripteurs`, **cocher** « Add a README file », **Create repository**.

2. **Settings**, **Collaborators**, **Add people**, le pseudo de Bob, puis celui de Chloé. GitHub peut redemander le mot de passe avant d'ouvrir la page.

Côté Bob et Chloé : accepter l'invitation, dans le mail ou à l'adresse `github.com/alice-martin/equipe-scripteurs/invitations`. Puis tout le monde clone :

Bob clone le dépôt d'équipe

```
$ git clone git@github.com:alice-martin/equipe-scripteurs.git
Cloning into 'equipe-scripteurs'...
done.
$ cd equipe-scripteurs
$ git log --oneline
3de79f2 (HEAD -> main, origin/main, origin/HEAD) Initial commit
$ ls
README.md
```

`Initial commit` est le commit créé par GitHub quand Alice a coché la case README. Cette fois, c'était voulu : personne n'avait d'historique local à pousser, tout le monde part de ce commit.

Vérification côté Alice : dans **Settings**, **Collaborators**, plus aucun « Pending invite ». Une invitation en attente, c'est un collaborateur qui ne pourra pas pousser.

Me2. Chacun sa branche, chacun sa PR

Bob ajoute son script dans sa branche

```
$ git switch -c ajout-bob
Switched to a new branch 'ajout-bob'
$ bash pair.sh 7
7 est impair
$ git add pair.sh
$ git commit -m "Ajoute le script pair.sh"
[ajout-bob 008f38f] Ajoute le script pair.sh
 1 file changed, 3 insertions(+)
 create mode 100644 pair.sh
$ git push -u origin ajout-bob
To git@github.com:alice-martin/equipe-scripteurs.git
 * [new branch]      ajout-bob -> ajout-bob
branch 'ajout-bob' set up to track 'origin/ajout-bob'.
```

Sur le vrai GitHub, le push affiche en plus une ligne `remote: Create a pull request for 'ajout-bob' on GitHub by visiting:` suivie d'un lien. Sur la page du dépôt, un bandeau propose **Compare & pull request**.

La pull request. Vérifiez en haut : `base: main` ← `compare: ajout-bob`. Un titre et une description utiles :

```
Titre : Ajoute le script pair.sh

Ce script dit si un nombre est pair ou impair.
Pour le tester : bash pair.sh 7   (doit afficher « 7 est impair »)
                 bash pair.sh 8   (doit afficher « 8 est pair »)
```

La relecture, par Chloé par exemple : onglet **Files changed**, elle teste le script sur sa machine, puis laisse au moins un commentaire, même positif (« Testé avec 7 et 8, ça marche. Peut-être afficher un message si on oublie l'argument ? »). Puis **Review changes**, **Approve**, **Submit review**. Une approbation sans commentaire ne prouve pas qu'elle a lu.

La fusion : Bob clique sur **Merge pull request**, **Confirm merge**. Quand les trois PR sont fusionnées, tout le monde récupère `main` :

Terminal

```
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
$ git pull
From github.com:alice-martin/equipe-scripteurs
   3de79f2..43009c8  main        -> origin/main
 * [new branch]      ajout-alice -> origin/ajout-alice
 * [new branch]      ajout-chloe -> origin/ajout-chloe
Updating 3de79f2..43009c8
Fast-forward
 pair.sh    | 3 +++
 rebours.sh | 2 ++
 table.sh   | 2 ++
 3 files changed, 7 insertions(+)
 create mode 100644 pair.sh
 create mode 100644 rebours.sh
 create mode 100644 table.sh
$ ls
README.md
pair.sh
rebours.sh
table.sh
```

Le `git switch main` dit *up to date with 'origin/main'* alors que trois PR ont été fusionnées : Git compare avec ce qu'il **sait** de GitHub depuis le dernier échange, pas avec GitHub en direct. Le `git pull` juste après récupère tout.

Me3. Le push refusé

Alice et Bob travaillent directement sur `main`, chacun sur un fichier différent. Alice pousse en premier :

Alice

```
$ echo "Équipe : Alice, Bob et Chloé" >> README.md
$ git commit -qam "Ajoute les membres de l'équipe"
$ git push
To git@github.com:alice-martin/equipe-scripteurs.git
   43009c8..93aec78  main -> main
```

Bob, juste après

```
$ sed -i 's/Dit si le nombre donné est pair/Dit si le nombre donné est pair ou impair/' pair.sh
$ git commit -qam "Précise le commentaire de pair.sh"
$ git push
To git@github.com:alice-martin/equipe-scripteurs.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'git@github.com:alice-martin/equipe-scripteurs.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

Le message à lire ensemble :
`Updates were rejected because the remote contains work that you do not have locally`. GitHub contient le commit d'Alice, que Bob n'a pas. S'il acceptait le push de Bob, le commit d'Alice serait perdu. Git refuse, et la solution est écrite juste en dessous : `git pull` d'abord.

Bob récupère, puis pousse

```
$ git pull
From github.com:alice-martin/equipe-scripteurs
   43009c8..93aec78  main       -> origin/main
Merge made by the 'ort' strategy.
 README.md | 1 +
 1 file changed, 1 insertion(+)
$ git log --oneline --graph -4
*   f88e78f (HEAD -> main) Merge branch 'main' of github.com:alice-martin/equipe-scripteurs
|\
| * 93aec78 (origin/main, origin/HEAD) Ajoute les membres de l'équipe
* | d579ca9 Précise le commentaire de pair.sh
|/
*   43009c8 Merge pull request #3 from alice-martin/ajout-chloe
|\
$ git push
To git@github.com:alice-martin/equipe-scripteurs.git
   93aec78..f88e78f  main -> main
```

Les deux commits ne touchent pas le même fichier : `git pull` les fusionne sans conflit, avec un commit de fusion (nano s'ouvre chez vous, on garde le message). Le push passe ensuite.

La leçon : **`git pull` avant de commencer à travailler**. Et pour ne plus avoir le problème du tout, chacun travaille sur sa branche, comme en Me2.

Me4. Le conflit à deux

Alice et Bob changent tous les deux la première ligne du README, dans leur branche. La PR d'Alice (`# Les Scripteurs`) est fusionnée en premier. Celle de Bob (`# Bash'n'Co`) passe en conflit sur GitHub. Bob règle le conflit sur sa machine :

Bob

```
$ git switch titre-bob
Switched to branch 'titre-bob'
Your branch is up to date with 'origin/titre-bob'.
$ git pull origin main
From github.com:alice-martin/equipe-scripteurs
 * branch            main       -> FETCH_HEAD
   f88e78f..3adf5a6  main       -> origin/main
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
$ cat README.md
<<<<<<< HEAD
# Bash'n'Co
=======
# Les Scripteurs
>>>>>>> 3adf5a6a138819742e39b9847128809a115de24c
Équipe : Alice, Bob et Chloé
```

L'autre version n'est pas étiquetée par un nom de branche mais par un identifiant de commit, une suite de chiffres et de lettres : c'est le commit de `main` que `git pull origin main` vient de récupérer. La résolution se fait de la même façon. Bob et Alice se mettent d'accord sur un titre qui réunit les deux idées, Bob l'écrit avec nano :

Bob règle le conflit

```
$ cat README.md
# Les Scripteurs de Bash'n'Co
Équipe : Alice, Bob et Chloé
$ git add README.md
$ git commit --no-edit
[titre-bob 7d6eb11] Merge branch 'main' of github.com:alice-martin/equipe-scripteurs into titre-bob
$ git push
To git@github.com:alice-martin/equipe-scripteurs.git
   709ad41..7d6eb11  titre-bob -> titre-bob
```

La ligne « Équipe : … » n'était pas dans le conflit : Git l'a gardée sans rien demander. Après le push, la PR de Bob se met à jour toute seule et le message de conflit disparaît.

> **Piège.** Régler le conflit sur `main` au lieu de sa branche. Le conflit se règle **dans la branche de la PR** : c'est elle qui doit contenir la résolution pour que la PR devienne fusionnable.

Me5. Relecture de code

Le script à relire, lancé avec trois notes :

moyenne_a_relire.sh

```bash
#!/bin/bash
# Affiche la moyenne et la meilleure des notes passées en arguments
notes=($@)
for n in "${notes[@]}"; do
    somme=$(( somme + n ))
done
moyenne=$(( somme / 5 ))
echo "Moyenne : $moyenne"
echo "Meilleure note : ${notes[0]}"
```

Exemple d'exécution

```
$ ./moyenne_a_relire.sh 12 8 15
Moyenne : 7
Meilleure note : 12
```

Les bonnes réponses sont 11 (35 / 3, en division entière) et 15. Les commentaires à laisser, chacun sur sa ligne dans **Files changed** :

Ligne	Commentaire de relecture
7 · `somme / 5`	**Bug.** Le nombre de notes est écrit en dur. Avec 3 notes, on divise par 5 et la moyenne est fausse (7 au lieu de 11). Proposition : `somme / ${#notes[@]}`.
9 · `${notes[0]}`	**Bug.** C'est la première note, pas la meilleure. Il manque la recherche du maximum dans la boucle.
5 · `somme=$((…))`	**Remarque.** `somme` n'est jamais initialisée. Bash la prend pour 0, ça marche ici, mais `somme=0` avant la boucle évite les surprises.
3 · `notes=($@)`	**Remarque.** Sans gravité pour des nombres, mais la bonne habitude est `notes=("$@")`, avec les guillemets.
tout le script	**Bonus.** Lancé sans argument, le script divise par zéro (avec la correction) : afficher un message d'usage.

Un bon commentaire de relecture dit **ce qui ne va pas**, **pourquoi**, et **propose** une solution. Puis **Request changes**. L'auteur corrige dans la même branche et pousse : la PR se met à jour, les relecteurs vérifient et approuvent.

La version corrigée, testée avec trois notes, puis sans argument :

moyenne_corrigee.sh

```bash
#!/bin/bash
# Affiche la moyenne et la meilleure des notes passées en arguments
notes=("$@")
if (( ${#notes[@]} == 0 )); then
    echo "Usage : bash moyenne_corrigee.sh note1 note2 ..."
    exit 1
fi
somme=0
max=${notes[0]}
for n in "${notes[@]}"; do
    somme=$(( somme + n ))
    if (( n > max )); then
        max=$n
    fi
done
moyenne=$(( somme / ${#notes[@]} ))
echo "Moyenne : $moyenne"
echo "Meilleure note : $max"
```

Exemple d'exécution

```
$ ./moyenne_corrigee.sh 12 8 15
Moyenne : 11
Meilleure note : 15
$ ./moyenne_corrigee.sh
Usage : bash moyenne_corrigee.sh note1 note2 ...
```

Me6. Issue fermée automatiquement

1. Onglet **Issues**, **New issue**, un titre par issue. Dans la colonne de droite, **Assignees** : la personne qui s'en charge.
2. Chacun crée sa branche, fait son travail, pousse et ouvre sa PR avec, dans la **description** : `Closes #1` (le numéro de son issue).
3. Après **Merge pull request**, l'issue passe toute seule en *Closed*, avec un lien vers la PR.

Ce qui fait marcher la fermeture automatique :

- le mot-clé est dans la **description** de la PR, ou dans un message de commit de la PR. Dans un simple commentaire, ça ne marche pas ;
- la PR est **fusionnée** dans la branche par défaut (`main`). Une PR fermée sans fusion ne ferme pas l'issue ;
- `Closes`, `Fixes` et `Resolves` fonctionnent tous les trois. Une PR peut fermer plusieurs issues : `Closes #1, closes #3`.

Sur la page de la PR, avant même la fusion, GitHub affiche l'issue liée dans la colonne de droite (**Development**). C'est le signe que le mot-clé a été compris.

Me7. Protéger main

Côté propriétaire : **Settings**, **Branches**, **Add classic branch protection rule** (ou **Add branch ruleset**, selon l'interface), motif `main`, cocher **Require a pull request before merging**, enregistrer.

Ensuite, un collaborateur qui pousse directement sur `main` reçoit un refus de ce type :

```
remote: error: GH006: Protected branch update failed for refs/heads/main.
remote: error: Changes must be made through a pull request.
To github.com:alice-martin/equipe-scripteurs.git
 ! [remote rejected] main -> main (protected branch hook declined)
error: failed to push some refs to 'github.com:alice-martin/equipe-scripteurs.git'
```

Avec un *ruleset*, le code est `GH013` et le message parle de *Repository rule violations*, mais le sens est le même : tout changement sur `main` doit passer par une PR. Le commit n'est pas perdu : il reste sur votre machine. Mettez-le dans une branche (`git switch -c ma-branche`, puis `git push -u origin ma-branche`) et ouvrez une PR.

> **Piège.** Le propriétaire du dépôt a les droits d'administrateur, et par défaut la règle ne s'applique pas à lui : son push direct peut passer. Faites l'essai depuis le compte d'un collaborateur, ou cochez **Do not allow bypassing the above settings** dans la règle.

JEUDI · GESTION DE PROJET ET AGILITÉ

Ici, pas de terminal : ce sont des exercices de réflexion. Les réponses proposées sont défendables, pas uniques. Ce qui compte, c'est la justification.

J1. Cascade ou agile ?

1. **Les fiches de paie et la nouvelle loi : plutôt cascade.** Le besoin est fixé par la loi, l'échéance aussi, et il faut pouvoir prouver que tout est conforme. On sait dès le départ ce qu'on doit livrer. L'équipe peut quand même avancer par petites étapes en interne.
2. **La start-up de partage de plantes : agile, sans hésiter.** Personne ne sait encore ce que les utilisateurs voudront vraiment. Il faut sortir une première version vite, observer comment elle est utilisée, et ajuster.
3. **Le site de la boulangerie : cascade, ou pas de méthode du tout.** Cinq pages, contenu prêt, besoin stable. Organiser des sprints pour ça, ce serait sortir un marteau-piqueur pour planter un clou.
4. **Le jeu en accès anticipé : agile.** Le principe même de l'accès anticipé, c'est de livrer souvent et d'écouter les joueurs.

La question à se poser : **le besoin est-il connu et stable ?** Si oui, la cascade marche bien. S'il va bouger, l'agilité est faite pour ça.

J2. Qui fait quoi ?

1. **Daily Scrum** : les trois questions du point quotidien (fait hier, fait aujourd'hui, ce qui bloque).
2. **Product Owner** : c'est lui qui décide de l'ordre du backlog.
3. **Sprint Review** : on montre ce qui est terminé aux utilisateurs, et leurs retours nourrissent le backlog.
4. **Scrum Master** : il s'occupe de la façon de travailler, pas du produit.
5. **Sprint Retrospective** : l'équipe améliore sa façon de travailler.
6. **Sprint Planning** : on choisit ce qu'on s'engage à terminer pendant le sprint.

Les confusions fréquentes : la Review regarde **le produit** (« qu'a-t-on construit ? »), la Rétrospective regarde **l'équipe** (« comment a-t-on travaillé ? »). Et le Scrum Master n'est pas un chef : il aide l'équipe, il ne lui donne pas d'ordres.

J3. Réparer des user stories

Story	Ce qui ne va pas (INVEST)	Une réécriture
« Faire la base de données. »	Une tâche technique, sans utilisateur ni valeur visible (**V**aluable).	*En tant qu'utilisateur, je veux retrouver mes tâches quand je relance l'application, afin de ne rien perdre.* La base de données devient un moyen.
« …que l'application soit bien. »	Impossible à estimer ou à tester (**E**stimable, **T**estable). Que veut dire « bien » ?	*En tant qu'utilisateur, je veux voir la liste de mes tâches en moins d'une seconde, afin de ne pas attendre.*
« …un bouton rouge en haut à droite. »	C'est une solution, pas un besoin (**N**egotiable), et il manque le « afin de ».	*En tant que joueur, je veux pouvoir quitter la partie à tout moment, afin de ne pas être obligé de la finir.*
« …gérer, partager, exporter, rappels, statistiques. »	Cinq besoins en un : trop gros (**S**mall), impossible à estimer.	Cinq stories séparées, une par besoin.
« En tant que développeur, je veux des tableaux associatifs. »	Un choix technique, aucune valeur pour l'utilisateur.	Pas une story : c'est une décision de l'équipe, prise en codant une vraie story.

Le réflexe : **qui** en a besoin, **quoi** exactement, et **pourquoi**. Si la story ne parle pas de l'utilisateur, ou si on ne sait pas dire quand elle est terminée, il faut la réécrire.

J4. Critères d'acceptation

Story 1 · proposer une lettre

- **Cas normal, bonne lettre** : *Étant donné* que le mot à trouver est « boucle », *quand* je propose « o », *alors* le mot s'affiche `_ o _ _ _ _` et mon nombre d'erreurs ne change pas.
- **Cas normal, mauvaise lettre** : *Étant donné* que le mot est « boucle », *quand* je propose « z », *alors* le jeu affiche « Raté » et mon nombre d'erreurs augmente de 1.
- **Cas limite** : *Étant donné* une partie en cours, *quand* je tape « AB » ou rien du tout, *alors* le jeu refuse la saisie et ne compte pas d'erreur.

Story 2 · savoir quand on a perdu

- **Cas normal** : *Étant donné* que j'ai fait 6 erreurs, *quand* je propose une lettre absente du mot, *alors* le jeu affiche « Perdu », montre le mot à trouver, et la partie s'arrête.
- **Cas limite** : *Étant donné* que j'ai fait 6 erreurs, *quand* je propose une lettre qui complète le mot, *alors* j'ai gagné, pas perdu.

Un bon critère se teste sans discussion : on lance le jeu, on fait exactement ce qui est écrit, et on voit si le résultat correspond. « Le jeu marche bien » n'est pas un critère.

J5. Planning poker

Il n'y a pas de bonne réponse : une estimation appartient à l'équipe qui va faire le travail. À titre de comparaison, pour une équipe qui débute en bash :

Story	Ordre de grandeur	Pourquoi
Dessin ASCII du pendu	2	sept dessins à écrire, puis un `case` sur le nombre d'erreurs
Mot au hasard dans `mots.txt`	2	lire le fichier dans un tableau, tirer un indice avec `$RANDOM`
Proposer de rejouer	1 à 2	une boucle autour de la partie
Les 10 meilleurs scores	5 à 8	lire, ajouter, trier, couper, réécrire un fichier : beaucoup d'étapes
Niveaux de difficulté	3	dépend de ce qu'on entend par « difficile » : à préciser avec le PO
Mode deux joueurs	3	saisir le mot sans l'afficher (`read -s`), puis la partie normale

Le plus utile, ce sont les **écarts au premier vote**. Celui qui vote 2 pour les meilleurs scores et celui qui vote 8 n'ont pas imaginé le même travail. Ils s'expliquent, l'équipe revote, et tout le monde a appris quelque chose sur la story.

J6. MoSCoW

- **Must** : ajouter, lister, marquer comme terminée, l'aide. Sans l'une d'elles, l'outil ne sert à rien.
- **Should** : supprimer, statistiques. Très utiles, mais on peut vivre une journée sans.
- **Could** : trier par priorité, date limite. Du confort, si on a le temps.
- **Won't** (pas cette fois) : synchronisation Google Agenda, interface web. Beaucoup trop gros pour une journée, et hors du terminal.

Le MVP en une phrase : *ajouter des tâches, les voir et les cocher depuis le terminal.*

Le MVP, c'est le plus petit produit qui rend déjà service. Si vous hésitez entre Must et Should, posez la question : « un utilisateur s'en servirait-il sans ça ? »

J7. Préparer le sprint de demain

Ce que le formateur vérifiait avant 17 h, à reprendre pour tout projet :

- chaque issue est une user story complète, avec son « afin de » ;
- chaque story a au moins un critère d'acceptation testable ;
- les Must suffisent à faire un MVP qui fonctionne ;
- aucune story n'est estimée à plus de 8 points. Au-delà, on la découpe ;
- la story « Ajouter le squelette » existe et passe en premier ;
- le tableau a ses quatre colonnes, avec toutes les issues dedans ;
- la Definition of Done est écrite dans le README.

Un exemple d'issue bien rédigée, pour `todo.sh` :

```
Titre : Marquer une tâche comme terminée

En tant qu'utilisateur, je veux marquer une tâche comme terminée,
afin de suivre mon avancement.

Critères d'acceptation
- Étant donné la tâche 2 « Acheter du pain » à faire,
  quand je tape ./todo.sh terminer 2,
  alors ./todo.sh lister l'affiche avec [x].
- Étant donné 3 tâches, quand je tape ./todo.sh terminer 9,
  alors le script affiche « Tâche 9 introuvable » et ne modifie rien.

Estimation : 3 · Priorité : Must
```

Et une Definition of Done réaliste pour une équipe débutante :

```
Une story est terminée quand :
- le code est sur main, fusionné par une PR relue et approuvée ;
- tous ses critères d'acceptation ont été testés à la main ;
- les erreurs prévues (argument manquant, numéro invalide) affichent un message clair ;
- l'aide et le README sont à jour.
```

LE JEU DE L'ENQUÊTE

Le script `creer_enquete.sh` fabrique toujours le même dépôt, avec des auteurs et des dates fixés : **vos identifiants de commit sont les mêmes que ceux de ce corrigé**. Seuls les commits que vous créez vous-mêmes (le `revert`, le bonus) seront différents.

Mise en place

```
$ bash creer_enquete.sh
Dépôt créé dans le dossier « enquete ». À vous de jouer : cd enquete
$ cd enquete
```

Question 1 · Le tour du propriétaire

Terminal

```
$ git log --oneline
882a9df (HEAD -> main) Corrige le titre du README
39f355e Fait le ménage dans le dépôt
3c8c82e Ajoute l'aide
6da784c Retire le mot de passe du dépôt
6a1b8ac Améliore l'affichage du résultat
421c76d Ajoute la connexion à la base de données
e932f0b Ajoute la division
80114ea Ajoute la multiplication
b06e9de Ajoute la calculatrice (addition et soustraction)
f9ff198 Ajoute le README
$ git log --oneline | wc -l
10
$ git shortlog -sn HEAD
     4  Alice Durand
     3  Bob Leroy
     3  Chloé Martin
```

10 commits. Alice en a fait le plus, 4. `git shortlog -sn` compte les commits par auteur : `-s` pour le résumé chiffré, `-n` pour trier du plus grand au plus petit. Le `HEAD` à la fin lui dit quoi compter : sans lui, dans certains cas, la commande attend qu'on lui envoie un historique au clavier et semble bloquée.

Question 2 · Qui a fait quoi, et quand ?

Terminal

```
$ git log --format="%h %an %ad %s" --date=short
882a9df Chloé Martin 2026-10-21 Corrige le titre du README
39f355e Alice Durand 2026-10-20 Fait le ménage dans le dépôt
3c8c82e Bob Leroy 2026-10-19 Ajoute l'aide
6da784c Alice Durand 2026-10-16 Retire le mot de passe du dépôt
6a1b8ac Chloé Martin 2026-10-16 Améliore l'affichage du résultat
421c76d Bob Leroy 2026-10-15 Ajoute la connexion à la base de données
e932f0b Chloé Martin 2026-10-14 Ajoute la division
80114ea Bob Leroy 2026-10-13 Ajoute la multiplication
b06e9de Alice Durand 2026-10-12 Ajoute la calculatrice (addition et soustraction)
f9ff198 Alice Durand 2026-10-12 Ajoute le README
```

La division a été ajoutée par Chloé Martin le 14 octobre, dans le commit `e932f0b`. Le plus ancien est en bas : l'historique se lit de bas en haut.

Question 3 · Le bug de la multiplication

Terminal

```
$ bash calc.sh 6 x 3
Résultat : 6 x 3 = 9
$ git blame calc.sh
b06e9de0 (Alice Durand 2026-10-12 10:02:00 +0200  1) #!/bin/bash
b06e9de0 (Alice Durand 2026-10-12 10:02:00 +0200  2) # Calculatrice : bash calc.sh 6 x 3
b06e9de0 (Alice Durand 2026-10-12 10:02:00 +0200  3) a=$1
b06e9de0 (Alice Durand 2026-10-12 10:02:00 +0200  4) op=$2
b06e9de0 (Alice Durand 2026-10-12 10:02:00 +0200  5) b=$3
b06e9de0 (Alice Durand 2026-10-12 10:02:00 +0200  6)
3c8c82eb (Bob Leroy    2026-10-19 10:10:00 +0200  7) if [[ -z $b ]]; then
3c8c82eb (Bob Leroy    2026-10-19 10:10:00 +0200  8)     echo "Usage : bash calc.sh nombre1 opération
nombre2   (opérations : + - x /)"
3c8c82eb (Bob Leroy    2026-10-19 10:10:00 +0200  9)     exit 1
3c8c82eb (Bob Leroy    2026-10-19 10:10:00 +0200 10) fi
3c8c82eb (Bob Leroy    2026-10-19 10:10:00 +0200 11)
b06e9de0 (Alice Durand 2026-10-12 10:02:00 +0200 12) case $op in
b06e9de0 (Alice Durand 2026-10-12 10:02:00 +0200 13)     +) resultat=$(( a + b )) ;;
b06e9de0 (Alice Durand 2026-10-12 10:02:00 +0200 14)     -) resultat=$(( a - b )) ;;
6a1b8ace (Chloé Martin 2026-10-16 09:05:00 +0200 15)     x) resultat=$(( a + b )) ;;
e932f0be (Chloé Martin 2026-10-14 11:20:00 +0200 16)     /) resultat=$(( a / b )) ;;
b06e9de0 (Alice Durand 2026-10-12 10:02:00 +0200 17)     *) echo "Opération inconnue : $op"; exit
1 ;;
b06e9de0 (Alice Durand 2026-10-12 10:02:00 +0200 18) esac
6a1b8ace (Chloé Martin 2026-10-16 09:05:00 +0200 19) echo "Résultat : $a $op $b = $resultat"
$ git show 6a1b8ac
commit 6a1b8acefaf63ec4b09440331fa52251c50fc997
Author: Chloé Martin <chloe@boite.fr>
Date:   Fri Oct 16 09:05:00 2026 +0200

    Améliore l'affichage du résultat

diff --git a/calc.sh b/calc.sh
index 5499bfa..3c90b5f 100644
--- a/calc.sh
+++ b/calc.sh
@@ -7,8 +7,8 @@ b=$3
 case $op in
     +) resultat=$(( a + b )) ;;
     -) resultat=$(( a - b )) ;;
-    x) resultat=$(( a * b )) ;;
+    x) resultat=$(( a + b )) ;;
     /) resultat=$(( a / b )) ;;
     *) echo "Opération inconnue : $op"; exit 1 ;;
 esac
-echo "$a $op $b = $resultat"
+echo "Résultat : $a $op $b = $resultat"
```

La ligne fautive : `x) resultat=$(( a + b ))`. C'est une **addition** au lieu d'une multiplication, d'où 6 + 3 = 9. `git blame` montre qu'elle vient du commit `6a1b8ac`, de Chloé, le 16 octobre.

Le message annonce-t-il ce que le commit a fait ? Non. Il s'intitule « Améliore l'affichage du résultat », mais `git show` révèle qu'il change **aussi** le calcul de la multiplication. Personne ne pouvait trouver ce bug en lisant `git log` : le message cachait la moitié du commit.

Question 4 · Le voyage dans le temps

Terminal

```
$ git switch --detach 421c76d
HEAD is now at 421c76d Ajoute la connexion à la base de données
$ bash calc.sh 6 x 3
6 x 3 = 18
$ git status
HEAD detached at 421c76d
nothing to commit, working tree clean
$ git switch main
Previous HEAD position was 421c76d Ajoute la connexion à la base de données
Switched to branch 'main'
```

Avant le commit fautif, 6 × 3 donnait bien 18. N'importe quel commit entre `80114ea` (l'ajout de la multiplication) et `421c76d` (le dernier avant le bug) convient.

`HEAD detached at 421c76d` : vous regardez le passé, sans être sur une branche. Un commit fait là-bas n'appartiendrait à aucune branche et se perdrait facilement. `git switch main` vous ramène au présent.

Question 5 · Réparer

Terminal

```
$ git revert --no-edit 6a1b8ac
Auto-merging calc.sh
[main 3041e75] Revert "Améliore l'affichage du résultat"
 Date: Mon Oct 26 10:00:00 2026 +0100
 1 file changed, 2 insertions(+), 2 deletions(-)
$ bash calc.sh 6 x 3
6 x 3 = 18
```

Le calcul est juste, mais **le « Résultat : » au début de la ligne a disparu**. Pourquoi ? `git revert` annule **tout** le commit, et ce commit mélangeait deux changements : l'affichage (voulu) et le calcul (le bug). Avec deux commits séparés, on aurait pu annuler seulement le mauvais.

L'auteur aurait dû faire deux commits, un par changement : c'est la règle de l'exercice L3. L'autre réparation possible, sans `revert` : corriger la ligne à la main et faire un commit « Corrige la multiplication ».

Question 6 · Le mot de passe

Terminal

```
$ git log -S "DB_PASSWORD" --oneline
6da784c Retire le mot de passe du dépôt
421c76d Ajoute la connexion à la base de données
$ git show 421c76d:config.sh
#!/bin/bash
# Connexion à la base de données
DB_USER="admin"
DB_PASSWORD="Soleil2026!"
```

Le mot de passe est `Soleil2026!`. L'affaire n'est pas close : supprimer le fichier dans un nouveau commit ne l'efface pas de l'historique. N'importe qui ayant accès au dépôt peut le lire avec `git show`, et sur un dépôt public GitHub, c'est tout Internet.

Ce que l'équipe doit faire : **changer le mot de passe**, tout de suite. C'est la seule vraie mesure. Pour l'avenir : mettre `config.sh` dans le `.gitignore` dès le premier jour, et ne versionner qu'un `config.exemple.sh` sans secret, comme l'a fait Alice.

Question 7 · Le fichier disparu

Terminal

```
$ git log --diff-filter=D --name-only --oneline
39f355e Fait le ménage dans le dépôt
notes.txt
6da784c Retire le mot de passe du dépôt
config.sh
$ git show 39f355e~1:notes.txt
TODO :
- ajouter la division
- penser au cas de la division par zéro !
$ bash calc.sh 6 / 0
calc.sh: line 16: a / b : division by 0 (error token is "b ")
6 / 0 =
```

Deux commits ont supprimé un fichier : `6da784c` a supprimé `config.sh` (la question 6), et **`39f355e` a supprimé `notes.txt`, par Alice, le 20 octobre**, dans « Fait le ménage dans le dépôt ».

`39f355e~1` désigne le commit **juste avant** la suppression : c'est là que le fichier existait encore. Il rappelait de gérer la division par zéro. Le problème existe toujours : bash affiche une erreur, puis une ligne de résultat vide.

Bonus

Terminal

```
$ git switch -c division-par-zero
Switched to a new branch 'division-par-zero'
$ git diff
diff --git a/calc.sh b/calc.sh
index d539dae..c1d388f 100644
--- a/calc.sh
+++ b/calc.sh
@@ -13,7 +13,8 @@ case $op in
     +) resultat=$(( a + b )) ;;
     -) resultat=$(( a - b )) ;;
     x) resultat=$(( a * b )) ;;
-    /) resultat=$(( a / b )) ;;
+    /) if (( b == 0 )); then echo "Division par zéro impossible"; exit 1; fi
+       resultat=$(( a / b )) ;;
     *) echo "Opération inconnue : $op"; exit 1 ;;
 esac
 echo "$a $op $b = $resultat"
$ bash calc.sh 6 / 0
Division par zéro impossible
$ bash calc.sh 6 / 3
6 / 3 = 2
$ git commit -qam "Gère la division par zéro"
$ git switch main
Switched to branch 'main'
$ git merge division-par-zero
Updating 3041e75..4cce031
Fast-forward
 calc.sh | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
$ git branch -d division-par-zero
Deleted branch division-par-zero (was 4cce031).
```

La correction : avant de diviser, on vérifie que `b` ne vaut pas 0, et sinon on affiche un message clair et on s'arrête avec `exit 1`. On vérifie le cas d'erreur **et** le cas normal.

`Fast-forward` : `main` n'avait pas bougé depuis la création de la branche, donc `git log --graph` reste une ligne droite. Pour voir une vraie bifurcation, il faut un commit sur `main` entre-temps, comme en M2.

LE JEU DU FESTIVAL

Comme pour l'enquête, les identifiants des commits créés par le script sont les mêmes chez tout le monde. Ceux que vous créez (fusions, cherry-pick, tag) seront différents.

Mise en place

```
$ bash creer_festival.sh
Dépôt créé dans le dossier « festival ». À vous de jouer : cd festival
$ cd festival
$ bash ../verifier_festival.sh
Missions 2 et 7 · Le travail mis de côté
  [..] le travail mis de côté n'est pas encore revenu et commité sur main
Mission 3 · La tête d'affiche du samedi
  [..] samedi-soir n'est pas encore fusionnée dans main
Mission 4 · Le conflit de l'affiche
  [..] la branche affiche n'est pas fusionnée, ou le conflit n'est pas résolu comme demandé
Mission 5 · Le bon tarif, et seulement lui
  [..] il faut le tarif étudiant sur main, mais pas le pass VIP
Mission 6 · La branche disparue
  [..] buvette.txt n'est pas encore sur main
Mission 8 · La version 1.0
  [..] pas de tag v1.0 sur le dernier commit de main

Score : 0 / 6
```

Mission 1 · L'état des lieux

Terminal

```
$ git status --short
 M programme.txt
$ git branch
  affiche
* main
  samedi-soir
  tarifs
$ git log --oneline --graph --all
* 50d0c89 (tarifs) Ajoute un pass VIP (prix à valider)
* 651e70f Ajoute le tarif étudiant
* d87e35d (HEAD -> main) Précise l'avancement de l'affiche
| * 205ce14 (affiche) Annonce l'illustratrice de l'affiche
|/
| * 0fc679b (samedi-soir) Annonce la tête d'affiche du samedi
|/
* d65cacc Crée le programme du festival
$ git branch --no-merged
  affiche
  samedi-soir
  tarifs
```

Quatre branches : `main`, `affiche`, `samedi-soir` et `tarifs`. Les trois dernières ne sont **pas fusionnées** dans `main`. Il y a du **travail non commité** : `M programme.txt`. La branche `buvette` n'apparaît nulle part, même avec `--all` : c'est le mystère de la mission 6.

Mission 2 · Mettre son travail de côté

Terminal

```
$ git diff
diff --git a/programme.txt b/programme.txt
index 82e58f2..7a67627 100644
--- a/programme.txt
+++ b/programme.txt
@@ -1,7 +1,7 @@
 FESTIVAL DES ROCHES · 2026
 Affiche : en cours de dessin
 Vendredi 20 h : Les Gratteurs
-Vendredi 22 h : à confirmer
+Vendredi 22 h : Fanfare Pistache
 Samedi 20 h : Duo Mistral
 Samedi 22 h : à confirmer
 Tarifs : 15 € la soirée
$ git merge samedi-soir
error: Your local changes to the following files would be overwritten by merge:
        programme.txt
Please commit your changes or stash them before you merge.
Aborting
Merge with strategy ort failed.
$ git stash
Saved working directory and index state WIP on main: d87e35d Précise l'avancement de l'affiche
$ git status --short
$ git stash list
stash@{0}: WIP on main: d87e35d Précise l'avancement de l'affiche
```

Git refuse la fusion : `Your local changes … would be overwritten by merge`. La fusion modifierait `programme.txt`, qui contient du travail pas encore commité. `git stash` range ce travail dans un tiroir et remet le fichier dans l'état du dernier commit. `git status` est vide : le dossier est propre.

Mission 3 · La tête d'affiche du samedi

Terminal

```
$ git merge --no-edit samedi-soir
Auto-merging programme.txt
Merge made by the 'ort' strategy.
 programme.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
$ grep "Samedi 22 h" programme.txt
Samedi 22 h : Les Ondes Vives
```

Mission 4 · Le conflit de l'affiche

Terminal

```
$ git merge affiche
Auto-merging programme.txt
CONFLICT (content): Merge conflict in programme.txt
Automatic merge failed; fix conflicts and then commit the result.
$ cat programme.txt
FESTIVAL DES ROCHES · 2026
<<<<<<< HEAD
Affiche : en cours de dessin
=======
Affiche : dessinée par l'atelier Kraft
>>>>>>> affiche
Vendredi 20 h : Les Gratteurs
Vendredi 22 h : à confirmer
Samedi 20 h : Duo Mistral
Samedi 22 h : Les Ondes Vives
Tarifs : 15 € la soirée
```

Alice (sur `main`) et Chloé (sur `affiche`) ont modifié la même ligne. On garde la version de Chloé : avec nano, on supprime les trois marqueurs **et** la ligne d'Alice.

Terminal

```
$ head -n 3 programme.txt
FESTIVAL DES ROCHES · 2026
Affiche : dessinée par l'atelier Kraft
Vendredi 20 h : Les Gratteurs
$ grep -c "<<<<<<<" programme.txt
0
$ git add programme.txt
$ git commit --no-edit
[main feb1f37] Merge branch 'affiche'
```

`grep -c` compte les lignes qui contiennent un marqueur : 0, aucun oubli.

Mission 5 · Le bon tarif, et seulement lui

Terminal

```
$ git log --oneline tarifs -3
50d0c89 (tarifs) Ajoute un pass VIP (prix à valider)
651e70f Ajoute le tarif étudiant
d87e35d Précise l'avancement de l'affiche
$ git cherry-pick 651e70f
Auto-merging programme.txt
[main 4bf1611] Ajoute le tarif étudiant
 Author: Alice Durand <alice@festival.fr>
 Date: Wed Oct 21 09:40:00 2026 +0200
 1 file changed, 1 insertion(+)
$ tail -n 2 programme.txt
Tarifs : 15 € la soirée
Tarif étudiant : 8 €
```

Le bon commit est `651e70f`, « Ajoute le tarif étudiant ». Le commit copié garde son auteur (Alice) et son message, mais il reçoit un **nouvel identifiant** : c'est une copie, pas le même commit. La branche `tarifs` n'a pas bougé.

> **Piège.** Faire le cherry-pick de `50d0c89`, le pass VIP. Git s'arrête sur un conflit : ce commit a été écrit juste après la ligne du tarif étudiant, qui n'existe pas sur `main`. La sortie de secours : `git cherry-pick --abort` remet tout comme avant. Ensuite, cherry-pick du bon commit.

Mission 6 · La branche disparue

Terminal

```
$ git reflog | grep buvette
d87e35d HEAD@{4}: checkout: moving from buvette to main
20fa784 HEAD@{5}: commit: Ajoute la carte de la buvette
d87e35d HEAD@{6}: checkout: moving from main to buvette
$ git branch buvette 20fa784
$ git merge --no-edit buvette
Merge made by the 'ort' strategy.
 buvette.txt | 3 +++
 1 file changed, 3 insertions(+)
 create mode 100644 buvette.txt
$ ls
README.md
buvette.txt
programme.txt
```

Le reflog garde la trace du commit de Bob, `20fa784`, même si plus aucune branche ne pointe dessus. `git branch buvette 20fa784` recrée la branche sur ce commit, et on la fusionne normalement.

Les numéros `HEAD@{n}` du reflog dépendent de ce que vous avez fait avant : chez vous, ils seront différents. L'identifiant `20fa784`, lui, est le même pour tous.

Mission 7 · Ressortir son travail

Terminal

```
$ git stash pop
Auto-merging programme.txt
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   programme.txt

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (c9e9942575ba3ae77c92704592c37059add2bd6f)
$ git diff
diff --git a/programme.txt b/programme.txt
index 577b972..85290da 100644
--- a/programme.txt
+++ b/programme.txt
@@ -1,7 +1,7 @@
 FESTIVAL DES ROCHES · 2026
 Affiche : dessinée par l'atelier Kraft
 Vendredi 20 h : Les Gratteurs
-Vendredi 22 h : à confirmer
+Vendredi 22 h : Fanfare Pistache
 Samedi 20 h : Duo Mistral
 Samedi 22 h : Les Ondes Vives
 Tarifs : 15 € la soirée
$ git commit -qam "Annonce la Fanfare Pistache"
```

`git stash pop` ressort le travail du tiroir et vide le tiroir. Pas de conflit : la fanfare est sur une ligne que personne d'autre n'a touchée. Le message de commit dit **ce que fait** la modification, pas « modif » ou « stash ».

Mission 8 · La version 1.0

Terminal

```
$ git tag -a v1.0 -m "Programme envoyé à l'imprimeur"
$ git tag
v1.0
$ cat programme.txt
FESTIVAL DES ROCHES · 2026
Affiche : dessinée par l'atelier Kraft
Vendredi 20 h : Les Gratteurs
Vendredi 22 h : Fanfare Pistache
Samedi 20 h : Duo Mistral
Samedi 22 h : Les Ondes Vives
Tarifs : 15 € la soirée
Tarif étudiant : 8 €
$ git log --oneline --graph
* 6eb0a68 (HEAD -> main, tag: v1.0) Annonce la Fanfare Pistache
*   3df0402 Merge branch 'buvette'
|\
| * 20fa784 (buvette) Ajoute la carte de la buvette
* | 4bf1611 Ajoute le tarif étudiant
* |   feb1f37 Merge branch 'affiche'
|\ \
| * | 205ce14 (affiche) Annonce l'illustratrice de l'affiche
* | |   73dbdbc Merge branch 'samedi-soir'
|\ \ \
| |_|/
|/| |
| * | 0fc679b (samedi-soir) Annonce la tête d'affiche du samedi
| |/
* / d87e35d Précise l'avancement de l'affiche
|/
* d65cacc Crée le programme du festival
$ bash ../verifier_festival.sh
Missions 2 et 7 · Le travail mis de côté
  [OK] plus rien dans le stash, la fanfare est commitée
Mission 3 · La tête d'affiche du samedi
  [OK] samedi-soir est fusionnée dans main
Mission 4 · Le conflit de l'affiche
  [OK] conflit résolu avec la version de Chloé
Mission 5 · Le bon tarif, et seulement lui
  [OK] tarif étudiant présent, pass VIP absent
Mission 6 · La branche disparue
  [OK] la carte de la buvette est sur main
Mission 8 · La version 1.0
  [OK] le tag v1.0 pointe sur le dernier commit de main

Score : 6 / 6
Bravo, le festival peut ouvrir ses portes !
```

Le tag `v1.0` apparaît dans le graphe à côté de `HEAD -> main`. Contrairement à `main`, il ne bougera plus : dans six mois, `git show v1.0` montrera toujours exactement ce qui est parti chez l'imprimeur.

Pour aller plus loin, les réponses

- Plusieurs `git stash` s'empilent : `stash@{0}` est le plus récent, et `git stash pop` ressort toujours le dernier entré.
- Par défaut, Git garde les entrées du reflog 90 jours, et 30 jours pour les commits qu'aucune branche n'atteint plus. Le reflog est **local** : il n'existe que sur votre machine, il ne part pas sur GitHub.

- Un `git push` simple n'envoie **pas** les tags. Il faut `git push origin v1.0`, ou `git push --tags` pour tous les envoyer.